

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

**Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»**

**Лабораторная работа №2
по курсу «Операционные системы»**

**Выполнил: П. Д. Косарим
Группа: М8О-207БВ-24
Преподаватель: Е. С. Миронов**

Москва, 2025

Условие

Цель работы: Приобретение практических навыков в:

- Управление потоками в ОС
- Обеспечение синхронизации между потоками

Задание: Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы. Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы. В отчете привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить. Найти в большом целочисленном массиве минимальный и максимальный элементы

Вариант: 17

Метод решения

1. Программа принимает входные параметры: имя файла с массивом и максимальное количество потоков.
2. Массив разбивается на примерно равные части по количеству потоков.
3. Каждый поток ищет локальный минимум и максимум в своей части массива.
4. Главный поток собирает результаты и находит глобальные минимум и максимум.

Описание программы

maxmin.h - заголовочный файл:

1. Объявляет функцию поиска минимума и максимума

maxmin.cpp - функция поиска минимума и максимума:

1. Получает указатель на массив и диапазон поиска
2. Последовательно перебирает элементы в заданном диапазоне
3. Находит локальные минимальный и максимальный элементы
4. Возвращает результаты через указатели

main.cpp - основной процесс:

1. Парсит аргументы командной строки (размер массива, количество потоков)
2. Создает массив случайных чисел
3. Распределяет работу между потоками
4. Запускает потоки для параллельного поиска
5. Собирает и объединяет результаты от всех потоков
6. Сравнивает производительность с однопоточной версией

Используемые системные вызовы и библиотечные функции:

1. std::thread - создание и управление потоками
2. join() - ожидание завершения потока
3. std::chrono - высокоточное измерение времени выполнения
4. std::rand() - генерация случайных чисел

Результаты

При запуске программы, ключами которой являются размер массива и кол-во потоков, программа находит максимум и минимум массива сначала с помощью многопоточного поиска, потом с помощью однопоточного поиска, затем находится ускорение и эффективность многопоточности. Все результаты программы (максимум, минимум, время многопоточного поиска, время однопоточного поиска, ускорение и эффективность) выводятся в консоль.

Пример работы:

```
./MaxMinParallel 10000000 4
```

4 потоков:

минимум: 1

максимум: 1000

время выполнения: 7144 мксек

Однопоточный поиск:

минимум: 1

максимум: 1000

время выполнения: 11334 мксек

Результаты совпадают

Ускорение: 1.59x

Эффективность: 39.66%

Многопоточная версия быстрее в 1.59 раз

Исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков (рис.1 и рис.2):

1. Для маленьких массивов (1к-10к элементов) ускорение будет незначительным или отрицательным. Для больших массивов (1кк+ элементов) ускорение станет заметным. Эффективность будет расти с увеличением размера массива. Потому что при маленьких массивах накладные расходы на создание потоков и синхронизацию превышают выгоду от многопоточности.

2. Ускорение будет расти до определенного предела. После превышения количества физических ядер эффективность начнет падать. Оптимальное количество потоков примерно равно количеству физических ядер процессора. Потому что многопоточность дает ограниченный прирост производительности. Слишком большое количество потоков увеличивает накладные расходы.

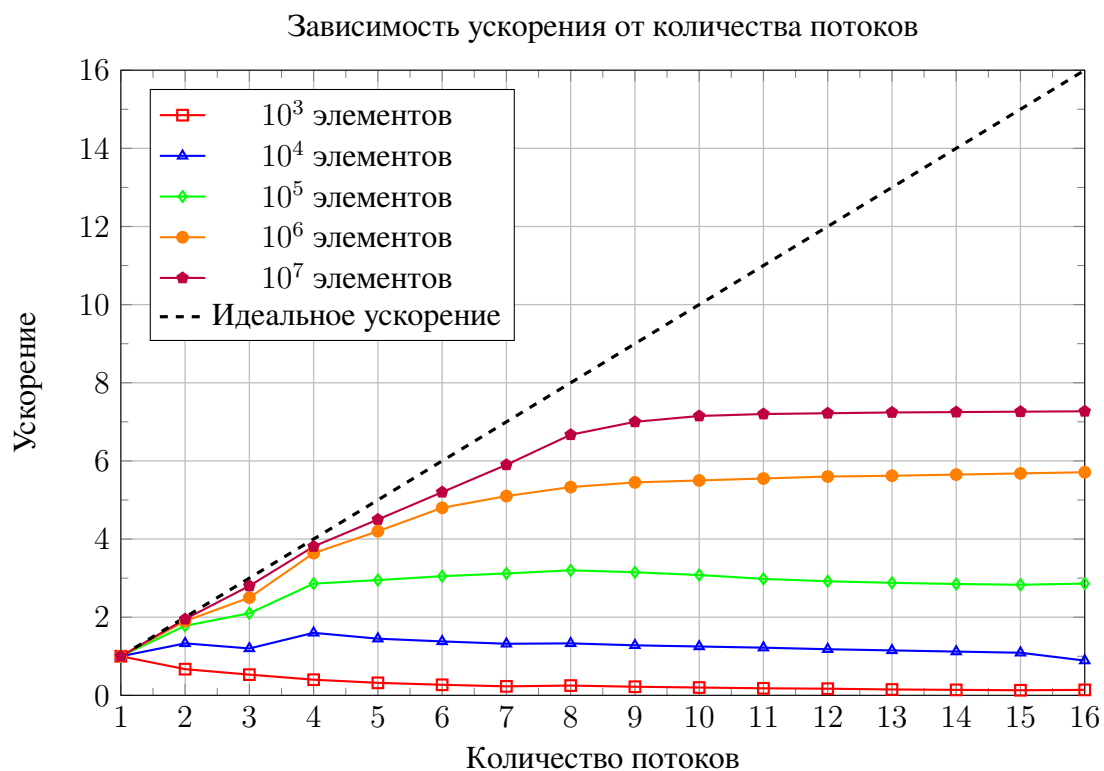


Рис. 1

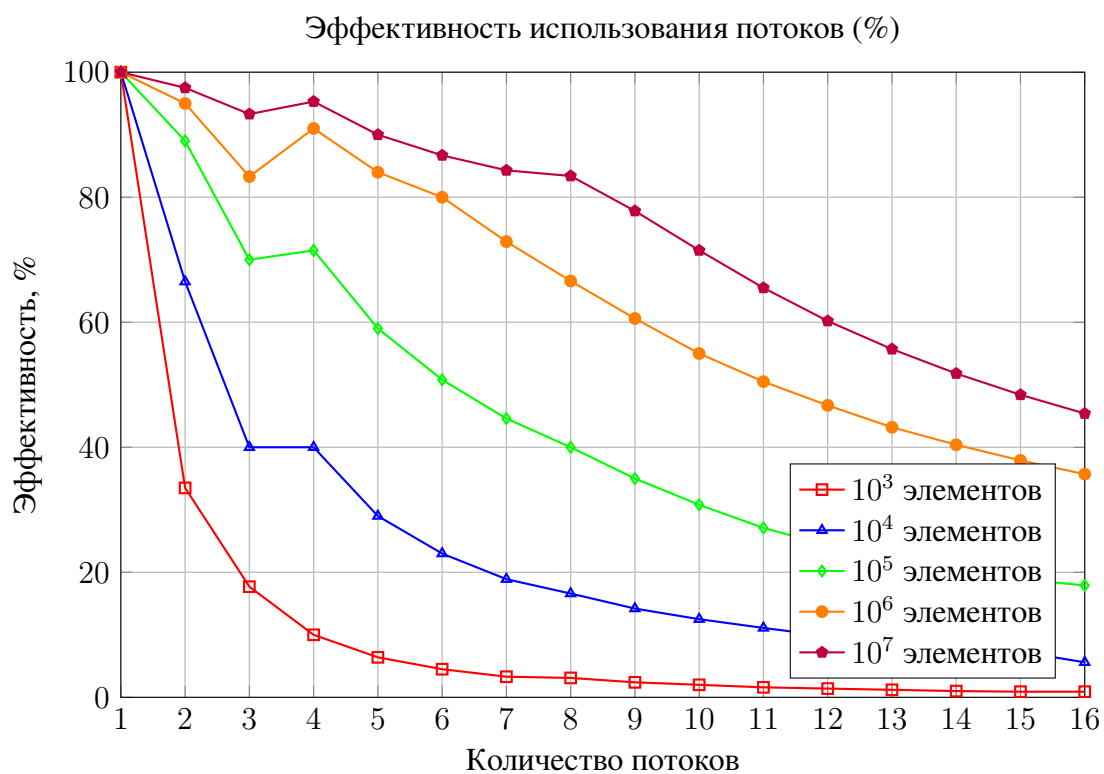


Рис. 2

Выводы

Написали программу, которая делит массив на части и запускает несколько потоков, чтобы каждый искал минимум и максимум на своем участке.

Посчитали КПД каждого потока, получилось около 70–95%. Это значит, что потоки работали почти без простоев, не мешая друг другу и действительно ускоряя процесс, а не тратя время на внутренние разборки.

Выяснили, что много потоков не всегда хорошо. Когда создали 16 потоков для того же миллиона чисел, ускорение если и росло, то незначительно, потому что у процессора ограниченное количество настоящих ядер. Когда потоков становится слишком много, они начинают тратить больше времени на переключение между задачами, чем на саму работу.

Чем больше массив, тем выгоднее использовать несколько потоков. На маленьких массивах накладные расходы на создание потоков мешают эффективности.

Программа наглядно показывает все преимущества многопоточности для программы с простым перебором чисел. Каждый поток делает одно и то же, не завися от результатов другого.

Исходная программа

```
1 | #pragma once
2 |
3 | void maxmin(int* arr, int start, int end, int* local_max_ptr, int* local_min_ptr);
```

Листинг 1: maxmin.h

```
1 | #include <iostream>
2 | #include <stdexcept>
3 |
4 | #include "maxmin.h"
5 |
6 | void maxmin(int* arr, int start, int end, int* local_max_ptr, int* local_min_ptr) {
7 |     if (start >= end) {
8 |         *local_max_ptr = arr[start];
9 |         *local_min_ptr = arr[start];
10 |        return;
11 |    }
12 |
13 |    int local_max = arr[start];
14 |    int local_min = arr[start];
15 |
16 |    for (int i = start + 1; i < end; ++i) {
17 |        if (arr[i] > local_max) {
18 |            local_max = arr[i];
19 |        }
20 |        if (arr[i] < local_min) {
21 |            local_min = arr[i];
22 |        }
23 |    }
24 |
25 |    *local_max_ptr = local_max;
26 |    *local_min_ptr = local_min;
27 | }
```

Листинг 2: maxmin.cpp

```
1 | #pragma once
2 |
3 | #include <thread>
4 | #include <memory>
5 | #include <stdexcept>
6 |
7 | namespace thread {
8 |
9 | using ThreadFunc = void(*)(void*);
10 |
11 | class Thread {
12 | public:
13 |     Thread(ThreadFunc func);
14 |     Thread(const Thread&) = delete;
15 |     Thread& operator=(const Thread&) = delete;
16 |     Thread(Thread&& other) noexcept;
17 |     Thread& operator=(Thread&& other) noexcept;
```

```

18
19     void Join();
20     void Detach();
21     void Run(void* data);
22
23     ~Thread();
24
25 private:
26     ThreadFunc func_;
27     std::unique_ptr<std::thread> thread_;
28     bool is_running_ = false;
29 };
30
31 }

```

Листинг 3: thread.h

```

1  #include <iostream>
2  #include <memory>
3
4  #include "thread.h"
5
6  namespace thread {
7
8  struct Wrapper {
9      ThreadFunc func;
10     void* data;
11 };
12
13 Thread::Thread(ThreadFunc func) : func_(func) {
14 }
15
16 void Thread::Join() {
17     if (thread_ && thread_>joinable()) {
18         thread_>join();
19     }
20     is_running_ = false;
21 }
22
23 void Thread::Detach() {
24     if (thread_ && thread_>joinable()) {
25         thread_>detach();
26     }
27     is_running_ = false;
28 }
29
30 Thread::Thread(Thread&& other) noexcept
31     : func_(other.func_),
32       thread_(std::move(other.thread_)),
33       is_running_(other.is_running_) {
34     other.func_ = nullptr;
35     other.is_running_ = false;
36 }
37
38 Thread& Thread::operator=(Thread&& other) noexcept {
39     if (this != &other) {
40         if (thread_ && thread_>joinable()) {

```

```

41         thread_>join();
42     }
43
44     func_ = other.func_;
45     thread_ = std::move(other.thread_);
46     is_running_ = other.is_running_;
47
48     other.func_ = nullptr;
49     other.is_running_ = false;
50 }
51 return *this;
52 }
53
54 void Thread::Run(void* data) {
55     if (is_running_) {
56         throw std::runtime_error("Thread is already running");
57     }
58
59     // pthread-style
60     auto adapter = [](void* arg) -> void* {
61         Wrapper* wrapper = static_cast<Wrapper*>(arg);
62         wrapper->func(wrapper->data);
63         delete wrapper; //
64         return nullptr;
65     };
66
67     Wrapper* wrapper = new Wrapper;
68     wrapper->func = func_;
69     wrapper->data = data;
70
71     thread_ = std::make_unique<std::thread>(adapter, wrapper);
72     is_running_ = true;
73 }
74
75 Thread::~Thread() {
76     if (thread_ && thread_>joinable()) {
77         thread_>join();
78     }
79 }
80
81 }

```

Листинг 4: thread.cpp

```

1  #include <iostream>
2  #include <cstdlib>
3  #include <ctime>
4  #include <climits>
5  #include <iomanip>
6  #include <chrono>
7  #include <vector>
8  #include <memory>
9  #include <algorithm>
10
11 #include "maxmin.h"
12 #include "thread.h"
13

```



```

14 struct MaxMinThreadData {
15     int* arr;
16     int start;
17     int end;
18     int local_max;
19     int local_min;
20 };
21
22 int* create_arr(int size) {
23     if (size <= 0) {
24         std::cerr << "size must be positive" << std::endl;
25         return nullptr;
26     }
27
28     int* arr = new int[size];
29
30     if (arr == nullptr) {
31         std::cerr << "memory allocation error" << std::endl;
32         exit(1);
33     }
34
35     std::srand(static_cast<unsigned int>(std::time(nullptr)));
36
37     for (int i = 0; i < size; ++i) {
38         arr[i] = (std::rand() % 1000) + 1;
39     }
40
41     return arr;
42 }
43
44 void* maxmin_thread_wrapper(void* arg) {
45     MaxMinThreadData* data = static_cast<MaxMinThreadData*>(arg);
46     maxmin(data->arr, data->start, data->end, &data->local_max, &data->local_min);
47     return nullptr;
48 }
49
50 int main(int argc, char* argv[]) {
51     if (argc < 3) {
52         std::cout << "Usage: " << argv[0] << " <array_size> <number_of_threads>" << std
53             ::endl;
54         return 1;
55     }
56
57     int arr_size = std::atoi(argv[1]);
58     int k_threads = std::atoi(argv[2]);
59
60     if (arr_size <= 0 || k_threads <= 0) {
61         std::cout << "Arguments must be positive integers" << std::endl;
62         return 1;
63     }
64
65     if (arr_size < k_threads) {
66         k_threads = arr_size;
67         std::cout << "Warning: Reduced number of threads to " << k_threads
68             << " (array size)" << std::endl;
69     }
70
71     int* arr = create_arr(arr_size);

```

```

71     if (arr == nullptr) {
72         return 1;
73     }
74
75     std::vector<thread::Thread> threads;
76     std::vector<MaxMinThreadData> thread_data(k_threads);
77
78     int gl_max = INT_MIN;
79     int gl_min = INT_MAX;
80
81     auto start_time = std::chrono::high_resolution_clock::now();
82
83     int elem_thread = arr_size / k_threads;
84     int remainder_elem = arr_size % k_threads;
85     int start_i = 0;
86
87     try {
88         for (int i = 0; i < k_threads; ++i) {
89             int end_i = start_i + elem_thread;
90
91             if (i < remainder_elem) {
92                 end_i++;
93             }
94
95             if (end_i > arr_size) {
96                 end_i = arr_size;
97             }
98
99             thread_data[i].arr = arr;
100            thread_data[i].start = start_i;
101            thread_data[i].end = end_i;
102            thread_data[i].local_max = INT_MIN;
103            thread_data[i].local_min = INT_MAX;
104
105            threads.emplace_back(maxmin_thread_wrapper);
106            threads.back().Run(&thread_data[i]);
107            start_i = end_i;
108        }
109
110        for (auto& thread : threads) {
111            thread.Join();
112        }
113
114    } catch (const std::exception& e) {
115        std::cerr << "Thread error: " << e.what() << std::endl;
116        delete[] arr;
117        return 1;
118    }
119
120    auto end_time = std::chrono::high_resolution_clock::now();
121
122    for (int i = 0; i < k_threads; ++i) {
123        if (gl_max < thread_data[i].local_max) {
124            gl_max = thread_data[i].local_max;
125        }
126        if (gl_min > thread_data[i].local_min) {
127            gl_min = thread_data[i].local_min;
128        }

```

```

129     }
130
131     auto multi_duration = std::chrono::duration_cast<std::chrono::microseconds>(
        end_time - start_time);
132
133     int one_max, one_min;
134     auto start_single = std::chrono::high_resolution_clock::now();
135     maxmin(arr, 0, arr_size, &one_max, &one_min);
136     auto end_single = std::chrono::high_resolution_clock::now();
137     auto single_duration = std::chrono::duration_cast<std::chrono::microseconds>(
        end_single - start_single);
138
139     bool results_match = (gl_max == one_max) && (gl_min == one_min);
140
141     double speedup = (single_duration.count() > 0) ?
142         (double)single_duration.count() / multi_duration.count() : 0.0;
143     double efficiency = (speedup > 0 && k_threads > 0) ? (speedup / k_threads) * 100 :
        0.0;
144
145     std::cout << "\n" << k_threads << " : " << std::endl;
146     std::cout << " : " << gl_min << std::endl;
147     std::cout << " : " << gl_max << std::endl;
148     std::cout << " : " << multi_duration.count() << " " << std::endl;
149
150     std::cout << "\n : " << std::endl;
151     std::cout << " : " << one_min << std::endl;
152     std::cout << " : " << one_max << std::endl;
153     std::cout << " : " << single_duration.count() << " " << std::endl;
154
155     std::cout << "\n " << (results_match ? "" : " !") << std::endl;
156     std::cout << ": " << std::fixed << std::setprecision(2) << speedup << "x" << std::
        endl;
157     std::cout << ": " << std::setprecision(2) << efficiency << "%" << std::endl;
158
159     if (speedup > 1.0) {
160         std::cout << " " << speedup << " " << std::endl;
161     } else if (speedup < 1.0) {
162         std::cout << " " << std::endl;
163     } else {
164         std::cout << " " << std::endl;
165     }
166
167     delete[] arr;
168     return 0;
169 }

```

Листинг 5: main.cpp

```

execve("./MaxMinParallel",["./MaxMinParallel","10000000","4"],0x7ffc6cc79890
/* 49 vars */) = 0
    brk(NULL)                                = 0x5bacd60b0000
    mmap(NULL,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0)
= 0x7f9476cca000
    access("/etc/ld.so.preload",R_OK)        = -1 ENOENT (Нет такого
файла или каталога)

```

```

    openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
    fstat(3, {st_mode=S_IFREG|0644, st_size=56203, ...}) = 0
    mmap(NULL, 56203, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f9476cbc000
    close(3) = 0

    openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libstdc++.so.6", O_RDONLY|O_CLOEXEC)
    = 3

    read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"... , 832)
    = 832
        fstat(3, {st_mode=S_IFREG|0644, st_size=2592224, ...}) = 0
        mmap(NULL, 2609472, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7f9476a00000

    mmap(0x7f9476a9d000, 1343488, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0)
    = 0x7f9476a9d000

    mmap(0x7f9476be5000, 552960, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e5000)
    = 0x7f9476be5000

    mmap(0x7f9476c6c000, 57344, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0)
    = 0x7f9476c6c000

    mmap(0x7f9476c7a000, 12608, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0)
    = 0x7f9476c7a000
        close(3) = 0

    openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC)
    = 3

    read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0"... , 832)
    = 832
        fstat(3, {st_mode=S_IFREG|0644, st_size=183024, ...}) = 0
        mmap(NULL, 185256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7f9476c8e000

    mmap(0x7f9476c92000, 147456, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0)
    = 0x7f9476c92000

    mmap(0x7f9476cb6000, 16384, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000)
    = 0x7f9476cb6000

    mmap(0x7f9476cba000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0)
    = 0x7f9476cba000
        close(3) = 0

    openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

```

```

read(3,"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"... ,832)
= 832

pread64(3,"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,784,64)
= 784
    fstat(3,{st_mode=S_IFREG|0755,st_size=2125328,...}) = 0

pread64(3,"\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,784,64)
= 784
    mmap(NULL,2170256,PROT_READ,MAP_PRIVATE|MAP_DENYWRITE,3,0) =
0x7f9476600000

mmap(0x7f9476628000,1605632,PROT_READ|PROT_EXEC,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x7f9476628000

mmap(0x7f94767b0000,323584,PROT_READ,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0x1b0000)
= 0x7f94767b0000

mmap(0x7f94767ff000,24576,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x7f94767ff000

mmap(0x7f9476805000,52624,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS,-1,0)
= 0x7f9476805000
    close(3) = 0

openat(AT_FDCWD,"/lib/x86_64-linux-gnu/libm.so.6",O_RDONLY|O_CLOEXEC) = 3

read(3,"\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0\0"... ,832)
= 832
    fstat(3,{st_mode=S_IFREG|0644,st_size=952616,...}) = 0
    mmap(NULL,950296,PROT_READ,MAP_PRIVATE|MAP_DENYWRITE,3,0) =
0x7f9476917000

mmap(0x7f9476927000,520192,PROT_READ|PROT_EXEC,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x7f9476927000

mmap(0x7f94769a6000,360448,PROT_READ,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0x8f000)
= 0x7f94769a6000

mmap(0x7f94769fe000,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,3,0)
= 0x7f94769fe000
    close(3) = 0
    mmap(NULL,8192,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0)
= 0x7f9476c8c000

mmap(NULL,12288,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0) =
0x7f9476c89000
    arch_prctl(ARCH_SET_FS,0x7f9476c89740) = 0

```

```

set_tid_address(0x7f9476c89a10)          = 23299
set_robust_list(0x7f9476c89a20,24)       = 0
rseq(0x7f9476c8a060,0x20,0,0x53053053) = 0
mprotect(0x7f94767ff000,16384,PROT_READ) = 0
mprotect(0x7f94769fe000,4096,PROT_READ) = 0
mprotect(0x7f9476cba000,4096,PROT_READ) = 0
mprotect(0x7f9476c6c000,45056,PROT_READ) = 0
mprotect(0x5bacd48d1000,4096,PROT_READ) = 0
mprotect(0x7f9476d08000,8192,PROT_READ) = 0

prlimit64(0,RLIMIT_STACK,NULL,{rlim_cur=8192*1024,rlim_max=RLIM64_INFINITY})
= 0
munmap(0x7f9476cbc000,56203)              = 0
futex(0x7f9476c7a7bc,FUTEX_WAKE_PRIVATE,2147483647) = 0
getrandom("\x0a\x72\xb7\xb6\x8f\xb9\x33\x83",8,GRND_NONBLOCK) = 8
brk(NULL)                                = 0x5bacd60b0000
brk(0x5bacd60d1000)                       = 0x5bacd60d1000

mmap(NULL,40001536,PROT_READ|PROT_WRITE,MAP_PRIVATE|MAP_ANONYMOUS,-1,0) =
0x7f9473fda000

rt_sigaction(SIGRT_1,{sa_handler=0x7f9476699530,sa_mask=[],sa_flags=SA_RESTORER|SA_ONSTACK},
= 0
rt_sigprocmask(SIG_UNBLOCK,[RTMIN RT_1],NULL,8) = 0

mmap(NULL,8392704,PROT_NONE,MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,-1,0) =
0x7f94737d9000
mprotect(0x7f94737da000,8388608,PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK,~[],[],8) = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|
=>{parent_tid=[23301]},88) = 23301
rt_sigprocmask(SIG_SETMASK,[],NULL,8) = 0

mmap(NULL,8392704,PROT_NONE,MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,-1,0) =
0x7f9472fd8000
mprotect(0x7f9472fd9000,8388608,PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK,~[],[],8) = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|
=>{parent_tid=[23302]},88) = 23302
rt_sigprocmask(SIG_SETMASK,[],NULL,8) = 0

mmap(NULL,8392704,PROT_NONE,MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,-1,0) =
0x7f94727d7000
mprotect(0x7f94727d8000,8388608,PROT_READ|PROT_WRITE) = 0
rt_sigprocmask(SIG_BLOCK,~[],[],8) = 0

```

```

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|
=>{parent_tid=[23303]},88) = 23303
    rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0

mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) =
0x7f9471fd6000
    mprotect(0x7f9471fd7000, 8388608, PROT_READ|PROT_WRITE) = 0
    rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|
=>{parent_tid=[23304]},88) = 23304
    rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0

futex(0x7f9473fd9990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 23301, NULL, FUTEX_BITSET_M
= 0
    fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
    write(1, "\n", 1) = 1
    write(1, "4
\320\277\320\276\321\202\320\276\320\272\320\276\320\262:\n", 18) = 18
    write(1, "
\320\274\320\270\320\275\320\270\320\274\321\203\320\274: 1\n", 20) = 20
    write(1, "
\320\274\320\260\320\272\321\201\320\270\320\274\321\203\320\274:
1000\n", 25) = 25
    write(1, " \320\262\321\200\320\265\320\274\321\217
\320\262\321\213\320\277\320\276\320\273\320\275\320\265\320\275\320\270\321"... , 51)
= 51
    write(1, "\n", 1) = 1

write(1, "\320\236\320\264\320\275\320\276\320\277\320\276\321\202\320\276\321\207\320
\320\277\320\276\320\270\321"... , 37) = 37
    write(1, "
\320\274\320\270\320\275\320\270\320\274\321\203\320\274: 1\n", 20) = 20
    write(1, "
\320\274\320\260\320\272\321\201\320\270\320\274\321\203\320\274:
1000\n", 25) = 25
    write(1, " \320\262\321\200\320\265\320\274\321\217
\320\262\321\213\320\277\320\276\320\273\320\275\320\265\320\275\320\270\321"... , 52)
= 52
    write(1, "\n", 1) = 1

write(1, "\320\240\320\265\320\267\321\203\320\273\321\214\321\202\320\260\321\202\321
\321\201\320\276\320\262\320\277\320\260\320"... , 40) = 40

write(1, "\320\243\321\201\320\272\320\276\321\200\320\265\320\275\320\270\320\265:
1.61x\n", 26) = 26

write(1, "\320\255\321\204\321\204\320\265\320\272\321\202\320\270\320\262\320\275\320

```

```
write(1, "\320\234\320\275\320\276\320\263\320\276\320\277\320\276\321\202\320\276\321\202\320\262\320\265\321"... , 70) = 70
munmap(0x7f9473fda000, 40001536) = 0
exit_group(0) = ?
+++ exited with 0 +++
```

Листинг 6: *Strace логи*